

ELEC ENG 2411 Ex: 3

Prepared by:
Lucas Rufkahr

Submitted to:
Sokrat Aldarmini

Submitted on:
September 21, 2026

Part 1 Step 1

We first create a vector T that runs from 0 to 10. We then execute a conditional statement $T==5$. This creates a variable `ans` with a type of vector. It has the same size of T except T is a double vector whereas `ans` is a logical vector. It is filled with zeroes except the position of 5 in T is equal to 1 in `ans`.

Part 1 Step 2

We create a new function called `unitStep()` in a new file named `unitStep.m`. This function takes a vector as an input and outputs another vector with the same dimensions. Where the values of t are greater than zero, we set the y vector to one and everywhere else to zero. This resulting output is a unit step function.

Part 1 Step 3

We first create a variable t using the `linspace()` function from -2 to 4 with 1000 points. We do this because we want to plot $x(t) = e^{-2t}(u(t) - u(t-1))$ and the 1000 points in between our interval will give us a smooth plot. We then create a variable $x = \exp(-2 * t) .* (\text{unitStep}(t) - \text{unitStep}(t - 1))$; We then use the functions `plot(t,x)`, `title('')`, `xlabel('')`, and `ylabel('')` to create the plot, set the title, and label the x and y axes. `exportgraphics()` was then used to render the figure to a PNG file. In figure 1 we can see the exponential function is zero for every t less than zero and greater than 1. The only region where the exponential behavior shows is from zero to one. Clearly we can see that the unit step function that was defined in Part 1 Step 2 is behaving normally.

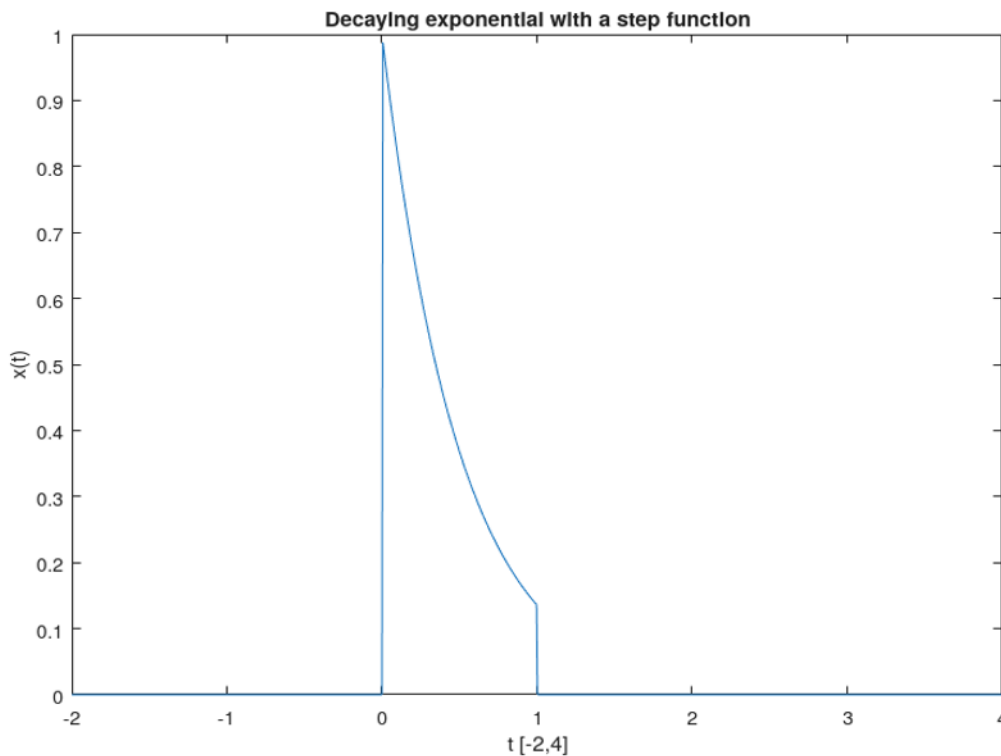


Figure 1: A decaying exponential multiplied by a step function from 0 to 1.

Part 1 Step 4

We now want to plot $x(t-1.5)$. To do this, we need to define a new variable `x_shift` which is essentially a copy of our original x except where t was, we replace with $(t - 1.5)$. We also want to plot this on top of our previous plot. To do this, we use the `hold on` statement and then we invoke the `plot(t,x_shift)` function. We now want to also plot $x(2t)$, and $x(-t)$ on top of the same figure as well which can be done using the same logic as before. To add a legend to our figure, we invoke the

`legend()` function. Labels are applied in the order the `plot()` function was called so we create the exponential first, shifted, compressed, and then reversed. The plot for this figure is shown in figure 2.

We can also calculate the energy of our functions. We use function `trapz()` which approximates the integral of a vector over an interval. We want to approximate the energy of $x(t)$, $x(t - 1.5)$, and $x(2t)$. To check the output of the function, we can also calculate this by hand. The formula for the trapezoidal method is equation 1 where $\Delta x = \frac{b-a}{n}$ and $x_i = a + i\Delta x$.

$$\int_a^b f(x)dx \approx \frac{\Delta x}{2} (f(x_0) + 2f(x_1) + 2f(x_2) + \dots + 2f(x_{n-1}) + f(x_n)) \quad (1)$$

To find the energy of a function, we need to take the absolute value of the function and square it. We also use the bounds of $(-\infty, \infty)$. Our equation becomes equation 2.

$$E = \int_{-\infty}^{\infty} |f(x)|^2 dx \approx \frac{\Delta x}{2} (|f(x_0)|^2 + 2|f(x_1)|^2 + 2|f(x_2)|^2 + \dots + 2|f(x_{n-1})|^2 + |f(x_n)|^2) \quad (2)$$

We begin by equating our bounds to a and b as it is where $x(\alpha x + \tau) > 0$ since everywhere else it is zero. For n , we will use 4 subintervals to preserve time. The energy equations are:

$$E_{x(t)} = \int_0^1 |x(t)|^2 dx \approx \frac{1}{8} (1 + 0.368 + 0.271 + 0.100 + 0.018) = 0.265 \quad (3)$$

$$E_{x(t-1.5)} = E_{x(t)} \quad (4)$$

$$E = \int_0^{0.5} |x(2t)|^2 dx \approx \frac{\Delta x}{2} (1 + 0.736 + 0.271 + 0.100 + 0.183) = 0.133 \quad (5)$$

Now that we've manually calculated the energies using a small resolution, we can see that our results are similar to but not exactly to the MATLAB calculated. Results are summarized in table 1.

Function	Energy (MATLAB)	Energy (Manual; n=4)
$x(t)$	0.24243	0.265
$x(t - 1.5)$	0.24695	0.265
$x(2t)$	0.11976	0.133

Table 1: Approximated integral using `trapz` compared to hand calculation with $n = 4$

To demonstrate why it is important to using a vector with a sufficiently large step size or resolution, we recreate our $x(t)$ using a time vector with a step size of one. The resulting energy when using the `trapz` function is near zero and is sufficiently smaller than our calculation using a time vector created with `linspace` and a resolution of 1000. Without a sufficiently large step size, the approximation is insufficiently accurate because a proper shape cannot be formed within the curve.

Now lets focus on the curve $x(-t)$. If we were to consider the output of a physical system, we need to make the assumption that the system is a causal system. The nature of a causal system is that for all $t < 0$, the output of the system must be zero. Now under the assumption that we are considering the output of a causal system, it then would not make sense if $x(-t)$ was the output because it does not agree with our criterion stated before.

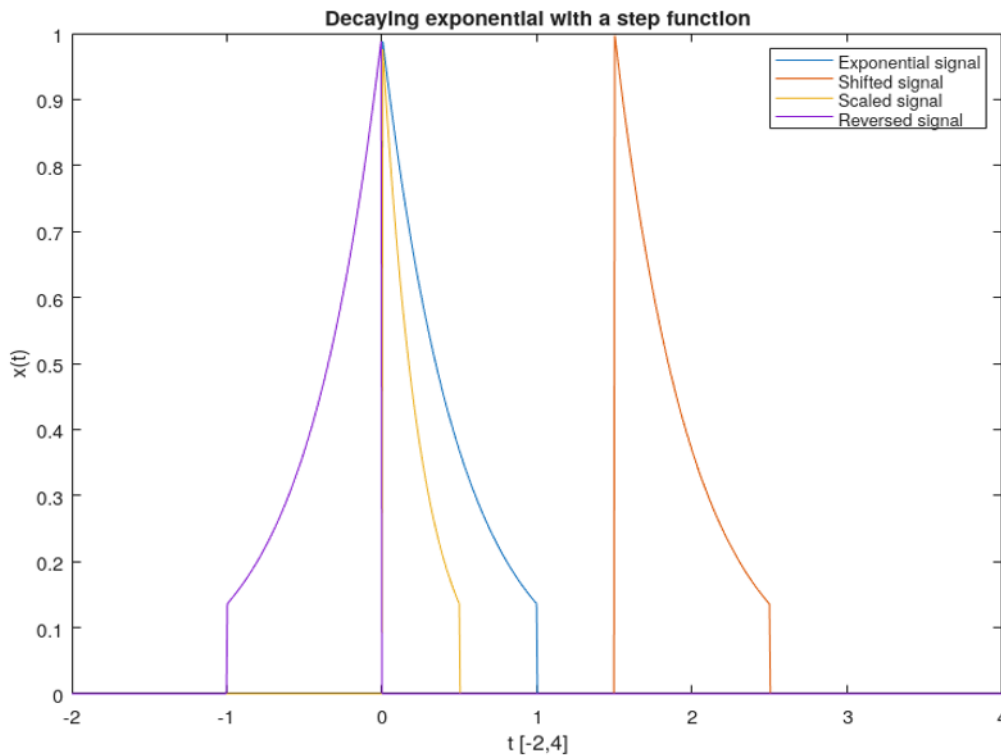


Figure 2: A plot of four curves, $x(t)$, $x(t - 1.5)$, $x(2t)$, and $x(-t)$.

Part 1 Step 5

This step concerns the `subplot()` function. We want each function: $x(t)$, $x(t - 1.5)$, $x(2t)$, $x(-t)$; to be plot on a new figure with each function on their own separate axis. We begin by creating a new figure with `figure()` and then calling `subplot(m, n, p)`. Since we have four plots, we want $m = n = 2$. We first call `subplot()` with $p = 1$ can then we call `plot()` for our $x(t)$. We then call our title and label functions to add them. Next with $p = 2$ we call `subplot()` again and then repeat the `plot()` function, title, and labels for $x(t - 1.5)$. Then we repeat this for the remaining functions with $p = 3$ and $p = 4$. The resulting figure is shown in figure 3.

We can also adjust the axis limits and enable the grid after we plot our functions just by calling `subplot()` to select the subplot we want to adjust. We use `xlim([i n])` to adjust the interval of the x-axis, and `grid on` to enable to grid. The figure with these extra modifications is shown in figure 4.

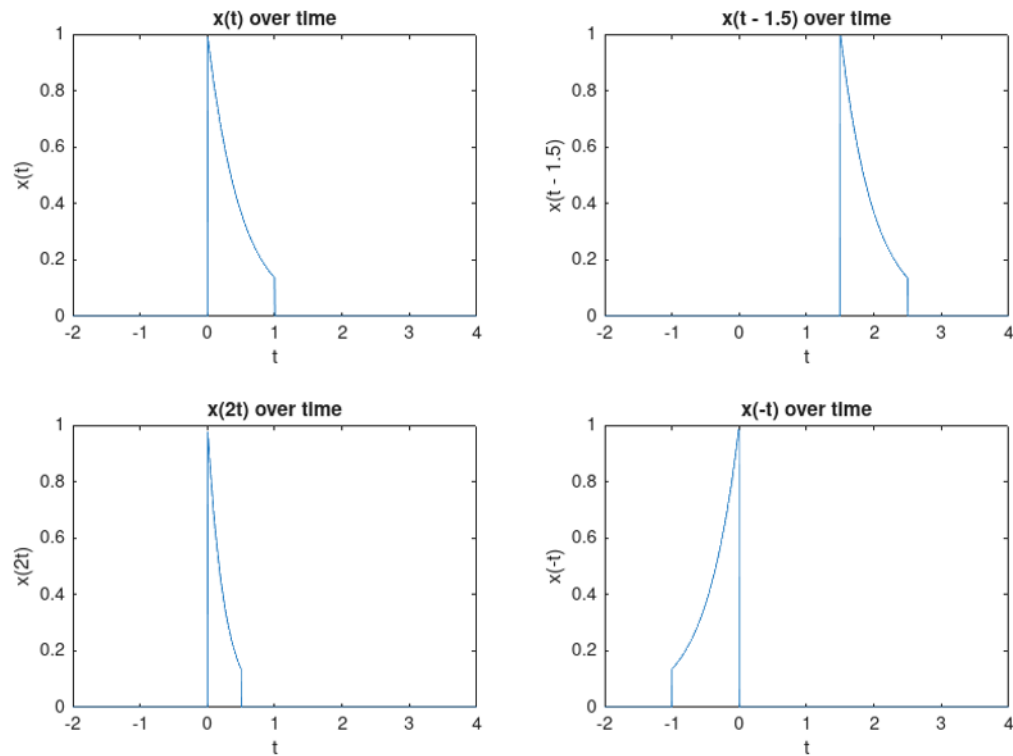


Figure 3: A figure containing four plots: $x(t)$, $x(t - 1.5)$, $x(2t)$, and $x(-t)$.

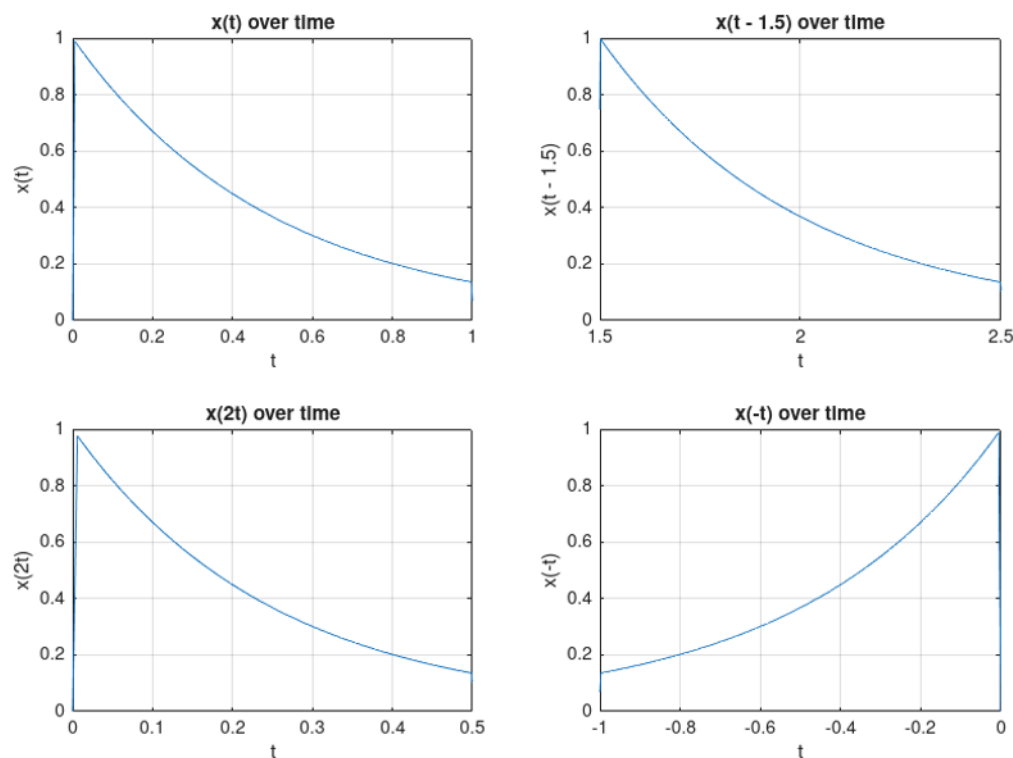


Figure 4: Modified figure 3 with each with a grid and adjusted x-axis interval.

Part 1 Step 6

In this step we will demonstrate how MATLAB can be used to interpret systems information as audio data. For this, we will consider the function $y(t) = \sin(2\pi(f_0t + 0.5\beta t^2))$ where $\beta = (f_1 - f_0)/T$, f_0 is the starting frequency, f_1 is the end frequency, and T is the duration. First we create a variable `freq_samp` equal to 441kHz. We then create a variable `T_samp` or the sampling period equal to one over the sampling frequency. Next we create our time domain vector which is a vector from zero to four with a step size equal to that of our sampling period. We will start with a frequency of 20Hz and work up to a frequency of 200Hz. We then create a variable `beta` and then define our function variable `y = sin(2 * pi * (freq_0 * t + 0.5 * beta * t.^2)) ;`. Next we create a new figure and plot our function `y`. We set the x-axis interval from 0 to 2 and our y-axis interval from -1.5 to 1.5. The resulting plot is shown in figure 5. We can see that the frequency increases as time increases. Using the `soundsc` function, we can play the sound and we hear that it is a sine wave increasing in frequency. To play this in reverse, we can use the `flipplr` function. If we play this reversed data, we can hear that it is a sine wave decreasing in frequency. We can also multiply `t` by two and divide `t` by two to create audio data that increases quicker and slower in frequency respectively. We can also create audio data that decreases in frequency and then increases by using concatenation. To do this, we define a new variable `fall_rise = [y_reversed, y]`. Playing this data, we can see that the frequency starts high, decreases, then rises again.

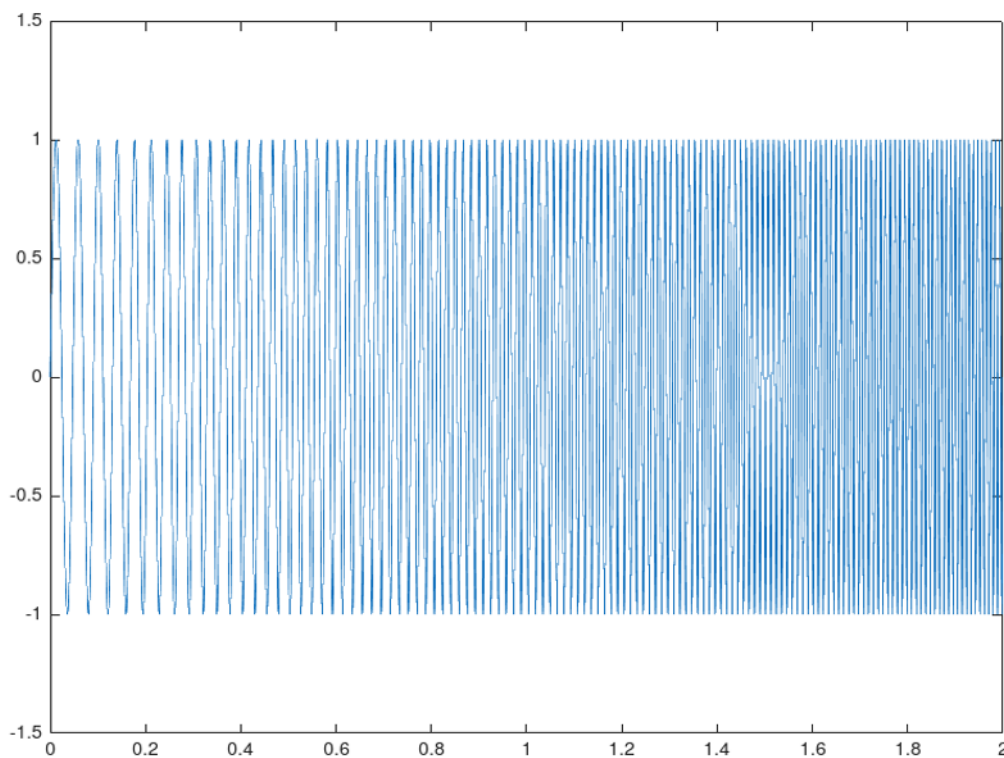


Figure 5: Audio signal starting at 20Hz and increasing linearly to 200Hz as time increases.

Part 2 Step 1

In part 2, we will work with three-dimensional data objects. We want to create a surface which we will do using the `surf()` function. First we define `x` and `y` vectors from -4 to 4 with a step size of 0.01. We create matrices which will serve as the `x` and `y` planes. We do this using the statement `[X, Y] = meshgrid(x, y)`. Next we create another matrix which will serve as the `Z` element in our three dimensions. We define `Z = 2 * cos(X) - sin(Y.^2)`. Now we use the `surf()` function with parameter `EdgeColor='none'`. We then add `x` and `y` labels to our figure. The resulting figure is shown in figure 6.

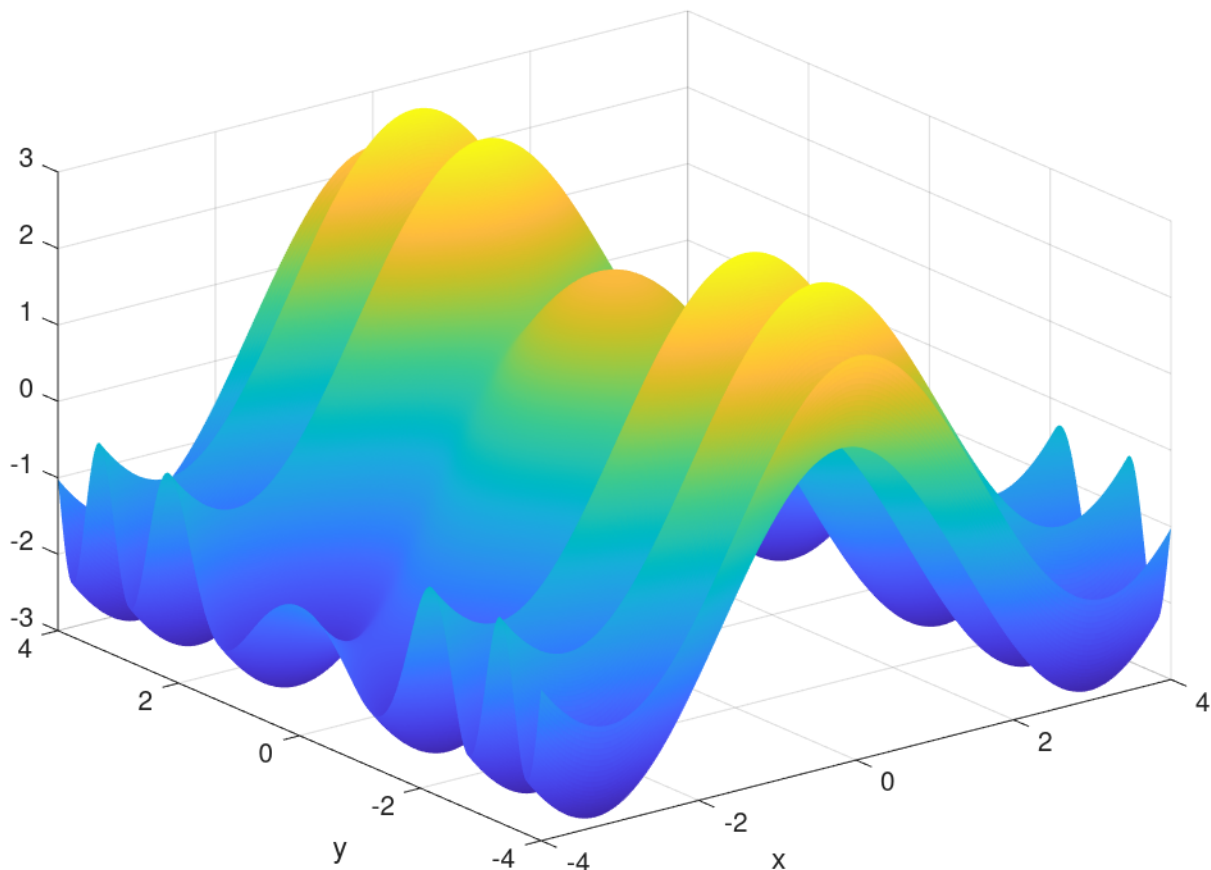


Figure 6: A three dimensional surface plot.

Part 2 Step 2

Now we will work with image data. In MATLAB, images are nothing but three dimensional objects. We will use stock MATLAB images and manipulate their data to give different results. First we load image data by defining a new variable `imData = imread('office_6.jpg')`. The function `imread()` will read the JPEG data from `office_6.jpg` and turn it into a three dimensional object that we can manipulate. We can show the figure using the `imshow()` function. The `imData` variable that was created contains pixel data in order of rows and columns. It also contains pages which correspond to individual color channels. We can show a red version of `office_6.jpg` by all color channels except red. We first create a new variable `imData1` as a copy of our original image data. Then we execute statements that set page two and three to zero. The resulting image is shown in figure 7. We can also remove color channel data such that the resulting image is yellow. To do this, we only remove the third color channel so that only red and blue remain. This is done by setting page three to zero. The image is shown in figure 8.



Figure 7: office_6.jpg with only red color channel data.

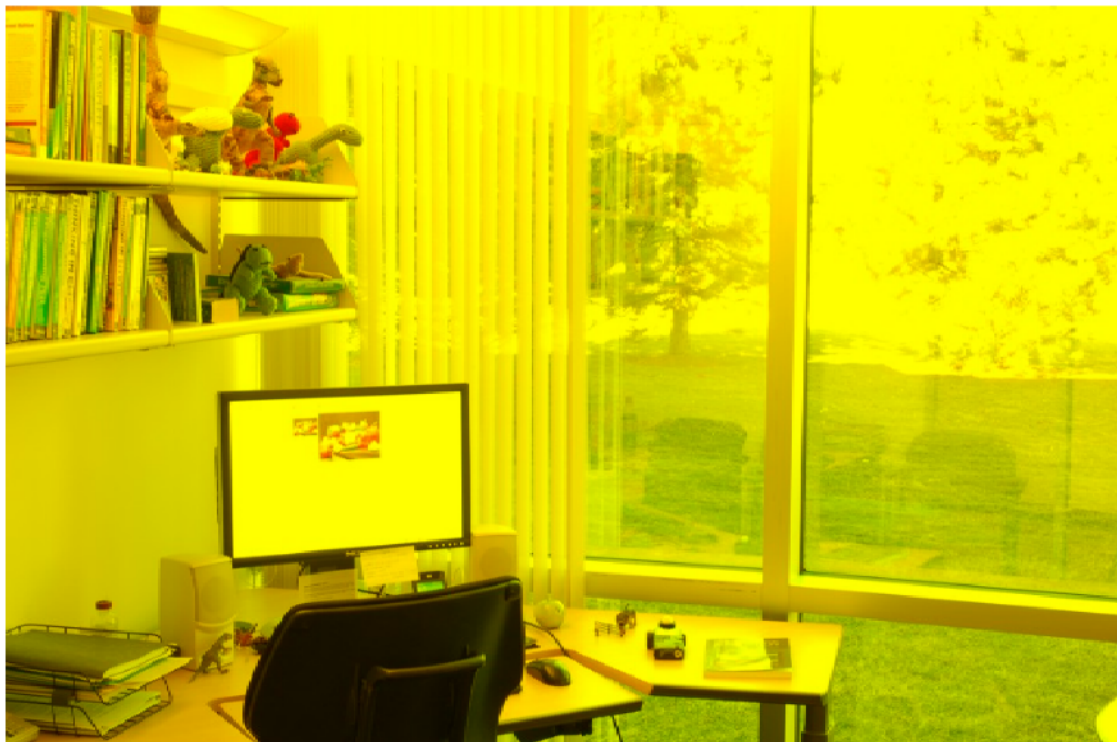


Figure 8: office_6.jpg with red and blue color channel data.

We can also manipulate the size of the image. This is done by only selecting rows and columns that one wishes to show. We will demonstrate this by showing only the bottom half and removing the end quarter of the image. To do this, we create a new variable `sliceImData = imread('office_6.jpg')`. Next we set our variable equal to only the data we require. Using the colon operator, we can select ranges of data. We do this by executing the statement `sliceImData =`

`sliceImData(end*0.5:end, 1:end*0.75, :)`. The first argument selects the row at the halfway point up to the last row. The second argument selects the first column up to 3/4 of all the columns. The last argument selects all the pages because we want to keep the color data as it originally is. The resulting image is shown in figure 9.

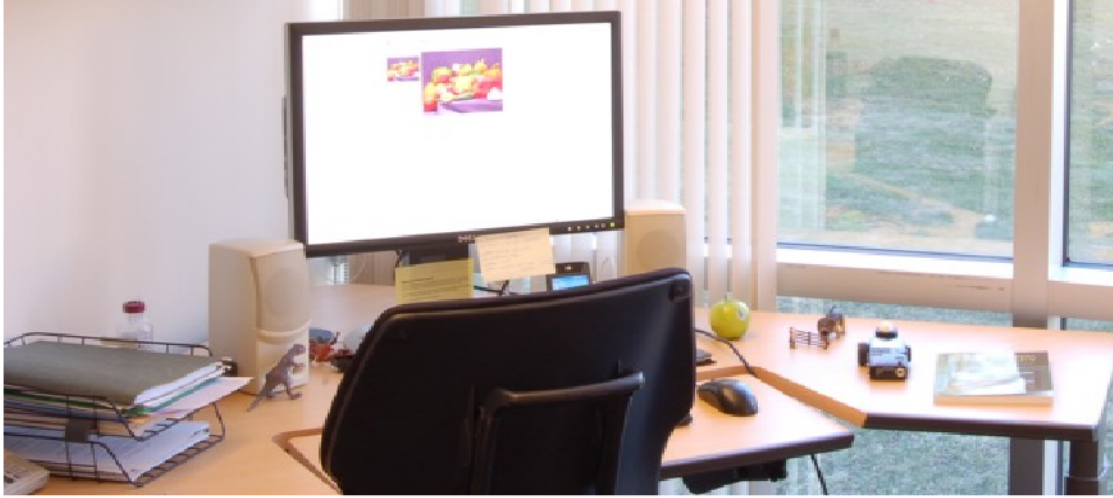


Figure 9: office_6.jpg with selectively chosen rows and columns.

We can also manipulate color data so that only brightness data is shown. We can do this by using the function `rgb2gray()`. We create a new variable `imDataG` equal to the grayscale function with image data as its only argument. The resulting image is shown in figure 10. We can also perform image normalization using equation 6.



Figure 10: office_6.jpg converted to grayscale.

$$\text{newData} = (\text{oldData} - \text{oldMin}) \times \frac{\text{newMax} - \text{newMin}}{\text{oldMax} - \text{oldMin}} + \text{newMin} \quad (6)$$

We first define `newMin = 2 * min(imDataGSF, [], 'all')` and `newMax = 0.5 * max(imDataGSF, [], 'all')` to create minimums that are twice as bright as the old minimums and maximums that are half as bright as the old maximums. The resulting image is shown in figure 11. A table showing minimum and maximum intensity before and after is shown in table 2.



Figure 11: office_6.jpg with image normalization applied.

	Minimum	Maximum
Old	8	255
New	16	127.5

Table 2: Minimum and maximum brightness data before and after normalization.